

```

from fastapi import FastAPI, UploadFile,
File, Form
from fastapi.responses import FileResponse
from typing import List
import io, re, os

app = FastAPI(title="Intelligent Resume
Screening")

SKILLS = [

    "python","java","javascript","typescript","
c++","c","sql","mongodb","mysql",
    "machine learning","deep
learning","artificial
intelligence","ai","pandas",
    "numpy","scikit-
learn","tensorflow","pytorch","fastapi","fl
ask","django",

    "git","github","docker","aws","azure","powe
r bi","excel","html","css",

    "react","node.js","nodejs","nlp","computer
vision","data analysis","rest api"
]

def extract_txt(data: bytes, filename: str)
-> str:
    ext =
os.path.splitext(filename.lower())[1]

```

```

        if ext == ".txt":
            return data.decode("utf-8",
errors="ignore")
        if ext == ".pdf":
            try:
                from pypdf import PdfReader
                reader =
PdfReader(io.BytesIO(data))
                return
"\n".join((p.extract_text() or "") for p in
reader.pages)
            except Exception:
                return ""
        if ext == ".docx":
            try:
                from docx import Document
                doc =
Document(io.BytesIO(data))
                return "\n".join(p.text for p
in doc.paragraphs)
            except Exception:
                return ""
        return ""

def normalize(s: str) -> str:
    return re.sub(r"^[a-z0-9+#.]+", " ",
s.lower()).strip()

def find_skills(text: str):
    n = normalize(text)
    found = []
    for skill in SKILLS:
        ns = normalize(skill)
        if re.search(r"(?<!\w)" +

```

```

re.escape(ns) + r"(?!\\w)", n):
    found.append(skill)
    return found

def candidate_name(text: str, filename:
str):
    lines = [x.strip() for x in
text.splitlines() if x.strip()]
    if lines and len(lines[0]) < 80:
        return lines[0]
    return os.path.splitext(filename)[0]

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/")
def home():
    return FileResponse("index.html")

@app.post("/screen")
async def screen(jd: str = Form(...),
resumes: List[UploadFile] = File(...)):
    required = find_skills(jd)
    results = []

    for resume in resumes:
        data = await resume.read()
        text = extract_txt(data,
resume.filename or "")
        matched = find_skills(text)
        matched_required = [s for s in
required if s in matched]
        missing = [s for s in required if s

```

```

not in matched]

        # Simple transparent scoring for a
demo:
    skill_score =
(len(matched_required) / len(required) *
85) if required else 0
    experience_bonus = 15 if
re.search(r"\b([2-9]|1[0-9])\+?\s*(years?
|yrs?)\b", text.lower()) else 0
    score = round(min(100, skill_score
+ experience_bonus), 1)

    results.append({
        "candidate":
candidate_name(text, resume.filename or
"Candidate"),
        "score": score,
        "matched_skills":
matched_required,
        "missing_skills": missing,
        "filename": resume.filename
    })

    results.sort(key=lambda x: x["score"],
reverse=True)
    for i, item in enumerate(results, 1):
        item["rank"] = i

    return {"results": results}

if __name__ == "__main__":
    import uvicorn

```

```
uvicorn.run("app:app",  
host="127.0.0.1", port=8000, reload=True)
```